Ma‘ruza 5: Ma’lumotlarni tozalash va tayyorlash

Missing values, dublikatlar, tiplar, outlierlar va matn tozalash

Sarvarbek Polvonov



Tashkent International University

Aniq fanlar va raqamli texnologiyalar kafedrasi

October 8, 2026

Bugungi ma’ruzadan ushbu bilimlarni olasiz

Ma’ruza yakunida siz quyidagilarni bajara olasiz

- Bo’sh qiymatlarni (`NaN`) topish, tahlil qilish va ular bilan ishlash strategiyasini tanlash
- Dublikat qatorlarni aniqlash va to’g’ri olib tashlash
- Ustun tiplarini to’g’irlash (`astype`)
- Chetdagi qiymatlarni (outlier) aniqlash va ular bilan nima qilishni hal qilish
- Matnni tozalash: `strip`, `replace`, regex
- Ma’lumot konsistentligini — mantiqiy izchillikni — tekshirish

Mavzuning muhimligi

Oldingi darsda ma’lumotni **ochib oldik**. Bugun uni **ishonchli** qilamiz — tahlil aynan shu bosqichda boshlanadi.

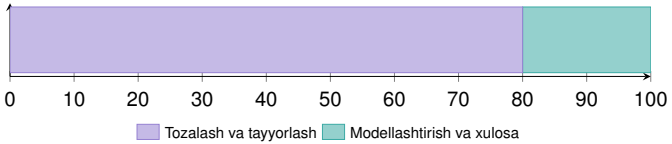
Ma’ruza rejasi

1. Nega tozalash dolzarb
2. Missing values bilan ishlash: dropna va fillna
3. Dublikat qatorlar
4. Ma’lumot tiplarini to’g’irlash
5. Outlierlarni aniqlash va tozalash
6. Matnni tozalash: strip, replace, regex
7. Konsistentlik: hammasini birlashtirish

Nega tozalash dolzarb

1. Nega tozalash dolzarb
2. Missing values bilan ishlash: dropna va fillna
3. Dublikat qatorlar
4. Ma’lumot tiplarini to’g’irlash
5. Outlierlarni aniqlash va tozalash
6. Matnni tozalash: strip, replace, regex
7. Konsistentlik: hammasini birlashtirish

80/20 qoidasi



Bu raqam qayerdan keldi

Ma’lumot fani jamoasida keng tarqalgan kuzatuv: loyihaning $\sim$**80%**i tozalash va tayyorlashga, faqat $\sim$**20%**i haqiqiy tahlilga ketadi. Aniq foiz loyihaga qarab farq qiladi, lekin tartib deyarli hamma joyda shunday.

Shuning uchun bu mavzu — alohida ma’ruza.

“Garbage in– garbage out”



Xavfli tomoni

Kod **xato bermaydi**. `df['narx'].mean()` tozalanmagan ma’lumotda ham chiroyli son qaytaradi — u shunchaki **noto’g’ri** bo’ladi. Tozalanmagan ma’lumotning eng katta xavfi — **sukut saqlab xato qilishi**.

“Garbage in, garbage out” — kompyuter fanidagi eng boshlang‘ich va eng haqiqiy qoidalardan biri.

Hayotiy misollar: tozalanmagan ma’lumot xarajati

1-holat: NASA Mars Climate Orbiter (1999)

Bitta jamoa metrik tizimda, boshqasi ingliz o’lchov birligida hisoblagan. Ma’lumotlar birlashtirilganda birlik nomuvofiqligi payqalmadi. Natija: $327 million qiymatidagi kosmik apparat yo’qoldi.

2-holat: saylov so’rovi xatosi (Literary Digest, 1936)

10 million kishiga so’rovnoma yuborildi, faqat telefon va avtomobil egalari javob berdi — o’sha davrda bu boy qatlam edi. Namuna katta bo’lsa ham **tizimli siljigan** edi, natija noto’g’ri bashorat berdi.

3-holat: ikki marta hisoblangan mijozlar

Kompaniya CRM’ida bir mijoz ikki xil email bilan ikki marta yozilgan. “Umumiy mijozlar soni” hisobotida raqam **shishirilgan** chiqdi — investorlarga noto’g’ri ma’lumot taqdim etilgan.

Umumiy o’xshashlik: muammo texnik emas edi — **tekshirilmagan taxmin** edi.

Tozalashning besh bosqichi



Ushbu besh bosqichni yo'l xaritasi bo'yicha davom ettiramiz

Har bo'lim: **muammoni ko'rish** $\rightarrow$ **nega paydo bo'lishini tushunish** $\rightarrow$ **pandas bilan yechish**.

Oltinchi bosqich — qolganlarni bog'laydigan

Konsistentlik tekshiruvi — besh muammoni birma-bir emas, **birgalikda** ko'rish.

NaN qayerdan paydo bo’ladi

```
mahsulot,narx,soni,izoh
Non,4500,120,
Sut,,45,yangi partiya
Daftar,8000,,
Ruchka,3000,200,yaxshi
```

...

	mahsulot	narx	soni	izoh
0	Non	4500.0	120.0	
1	Sut	NaN	45.0	yangi partiya
2	Daftar	8000.0	NaN	
3	Ruchka	3000.0	200.0	yaxshi

Bo’sh qiymatning kelib chiqishi sabalari turlicha:

- **Chindan yo’q** — savol tegishli emas (masalan, “turmush o’rtog’i ismi” bo’ydoq odam uchun)
- **O’lchanmagan** — sensor ishlamagan, so’rovnoma to’ldirilmagan
- **Yo’qolgan** — baza xatosi, qo’lda kiritishda tashlab ketilgan
- **Ataylab yashirilgan** — maxfiy ma’lumot, masalan maosh

Muhim savol: bo’sh qiymat **tasodifiymi** yoki **tizimlimi**? Javob tozalashga nisbatan keyingi strategiyani

NaN’ni aniqlash

```
df.isna() # har katak True/False
df.isna().sum() # har USTUNDA nechta bo'sh bor
df.isna().sum().sum() # JAMI nechta bo'sh qiymat

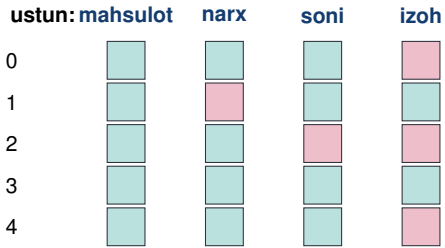
df.isna().mean() * 100 # har ustunda bo'shlik FOIZI

df.notna() # teskarisi -- to'la qiymatlar
df[df['narx'].isna()] # narxi bo'sh QATORLARNI ko'rish
```

Klassik xato: NaN == NaN natijasi False

`df[df['narx'] == None]` **ishlamaydi** — hech narsa topmaydi.
To’g’ri usul — **doim** `isna()` yoki `notna()` bilan tekshiring, tenglik bilan emas.

Bo’sh qiymatlarni vizual ko’rish



To’la va **bo’sh** kataklarni shu tarzda ko’rish naqshni ko’rsatadi: tasodifiymi yoki bitta ustunda to’planganmi?

```
import missingno as msno
msno.matrix(df) # xuddi shunday sxemani avtomatik chizadi
```

? Toping: bu ustunda nima noodatiy

```
df['yosh'].value_counts()
```

```
25 412
30 389
0 178 <- BU NIMA?
34 356
99 201 <- BU HAM?
```

? Toping: bu ustunda nima noodatiy

```
df['yosh'].value_counts()
```

```
25 412
30 389
0 178 <- BU NIMA?
34 356
99 201 <- BU HAM?
```

Yashirin NaN

0 va 99 — **yashirin bo'sh qiymatlar**. Eski tizimlarda "ma'lumot yo'q" holatini shunday kodlash odat bo'lgan (yosh uchun 0 yoki 99, koordinata uchun -999).

Yechim

```
df['yosh'] = df['yosh'].replace([0, 99], np.nan)
# yoki o'qishdayoq:
df = pd.read_csv('f.csv', na_values=[0, 99, -999, 'N/A'])
```

Uch turdagi “yo’qlik” – farqlar:

Tur	Ta’rif	Misol
MCAR	To’liq tasodifiy — hech narsaga bog’liq emas	Sensor tasodifiy vaqt uchun ishlamay qoldi
MAR	Boshqa **kuzatilgan** ustunga bog’liq	Yoshi kattalar maosh savoliga kamroq javob beradi
MNAR	**Qiymatning o’zi**ga bog’liq	Kam maosh oluvchilar buni yozishni istamaydi

Bu nazariy emas, amaliy bilim

MCAR bo’lsa — bo’sh qatorlarni tashlab yuborish xavfsiz.
MNAR bo’lsa — tashlab yuborish natijani **siljitib yuboradi** (masalan, past maoshlar tizimli tushib qoladi va o’rtacha maosh sun’iy oshib ketadi).

Missing values bilan ishlash: dropna va fillna

1. Nega tozalash dolzarb
2. Missing values bilan ishlash: dropna va fillna
3. Dublikat qatorlar
4. Ma’lumot tiplarini to’g’irlash
5. Outlierlarni aniqlash va tozalash
6. Matnni tozalash: strip, replace, regex
7. Konsistentlik: hammasini birlashtirish

dropna — olib tashlash

```
df.dropna() # NaN bo'lgan HAR QANDAY qator olib tashlanadi
df.dropna(subset=['narx']) # faqat 'narx' bo'sh bo'lsa olib tashlanadi
df.dropna(how='all') # QATOR TO'LIQ bo'sh bo'lsagina olib tashlanadi
df.dropna(thresh=3) # kamida 3 ta to'la qiymat bo'lsin

df.dropna(axis=1) # NaN bo'lgan USTUNLARNI olib tashlanadi
df.dropna(axis=1, thresh=len(df)*0.5) # 50%+ bo'sh ustunlar olib tashlanadi
```

Har doim tekshiring: qancha ma’lumot yo’qoldi?

```
oldin = len(df)
df = df.dropna(subset=['narx'])
print(f"{oldin - len(df)} qator olib tashlandi "
      f"({(oldin-len(df))/oldin:.1%})")
```

10%dan ko’p qator yo’qolsa — **boshqa usulni o’ylaymiz**, ehtimol `fillna` yaxshiroq.

fillna — to’ldirish

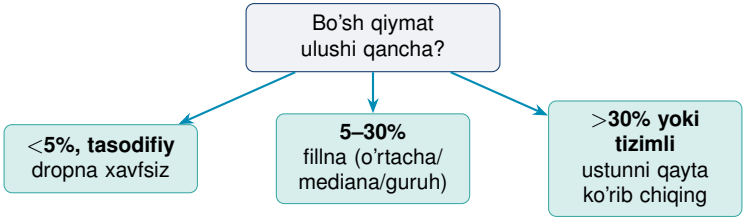
```
df['soni'].fillna(0) # doimiy qiymat
df['narx'].fillna(df['narx'].mean()) # o'rtacha bilan
df['narx'].fillna(df['narx'].median()) # mediana (chetki ta'sirga chidamli)
df['kategoriya'].fillna(df['kategoriya'].mode()[0]) # eng ko'p uchraydigan

# Guruh bo'yicha o'rtacha -- ancha aniqroq
df['narx'] = df.groupby('kategoriya')['narx'] \
              .transform(lambda x: x.fillna(x.mean()))

# Vaqt qatorlari uchun
df['harorat'].ffill() # oldingi qiymat bilan
df['harorat'].bfill() # keyingi qiymat bilan
df['harorat'].interpolate() # ikkisi orasidan chiziqli hisoblash
```

Ko’p qo’llaniladigan uslub: guruh bo’yicha to’ldirish — global o’rtacha emas, **mos guruh** o’rtachasi.

Qaysi strategiyani tanlash



Qo'shimcha ustun — kam ishlatiladigan, lekin kuchli usul

```
df['narx_bosh_edi'] = df['narx'].isna() # signal saqlanadi
df['narx'] = df['narx'].fillna(df['narx'].median())
```

Shunday qilib to'ldirilgan qiymat sun'iy ekanini keyingi tahlil ham **bilib turadi**.

Amaliy misol: talabalar davomati

```
davomat = pd.DataFrame({
    'talaba': ['Aziza','Bekzod','Dilnoza','Eldor','Feruza'],
    'dars1': [1, 1, np.nan, 1, 0],
    'dars2': [1, np.nan, 1, np.nan, 1],
    'dars3': [np.nan, 1, 1, 0, 1],
})

# Diagnostika
davomat.isna().sum()
davomat.isna().mean(axis=1) * 100 # har TALABA uchun bo'shlik foizi

# Qaror: dars qoldirilgan (0) deb hisoblaymiz -- domenga xos bilim
davomat_tozalangan = davomat.fillna(0)
```

Domenga xos bilim — eng muhim omil

Bu yerda `NaN` statistik emas, **ma'nodor**: "qatnashmagan" degani. Shuning uchun o'rtacha bilan to'ldirish emas, **0** bilan to'ldirish to'g'ri — vaziyatni bilmasdan strategiya tanlab bo'lmaydi.

Dublikat qatorlar

1. Nega tozalash dolzarb
2. Missing values bilan ishlash: dropna va fillna
3. Dublikat qatorlar
4. Ma’lumot tiplarini to’g’irlash
5. Outlierlarni aniqlash va tozalash
6. Matnni tozalash: strip, replace, regex
7. Konsistentlik: hammasini birlashtirish

Dublikatlar qayerdan keladi

Sabablari:

- Formani **ikki marta** yuborish (tugma qayta bosilgan)
- Ikki tizimdan olingan ma'lumotni **birlashtirish**
- API'dan **qayta so'rov** yuborilishi
- Excel'da qator **qo'lda ko'chirilgan**

Xavflari:

- `sum()`, `count()` — **shishiriladi**
- `mean()` qiyshayadi
- Guruhlashda noto'g'ri og'irlik
- "Umumiy mijozlar soni" xato chiqadi

```
df.duplicated() # har qator uchun True/False
df.duplicated().sum() # nechta dublikat bor
df[df.duplicated()] # dublikat qatorlarni ko'rish
```

To’liq va qisman dublikat

```
# TO'LIQ dublikat -- barcha ustunlar bir xil
df.duplicated()

# Faqat MUAYYAN ustunlar bo'yicha -- ko'proq ishlatiladi
df.duplicated(subset=['email'])
df.duplicated(subset=['ism', 'familiya', 'tugilgan_sana'])

# keep parametri -- QAYSI nusxa "asl" hisoblansin
df.duplicated(keep='first') # birinchisi asl, qolgani dublikat (standart)
df.duplicated(keep='last') # oxirgisi asl
df.duplicated(keep=False) # BARCHA nusxalar dublikat deb belgilanadi
```

Ko’p uchraydigan xato

Faqat `df.duplicated()` (parametrsiz) ishlatish — bu **barcha** ustun bo’yicha aynan bir xillikni talab qiladi. Amalda bitta ustun (masalan, `buyurtma_vaqti`) millisekundida farq qilib, dublikat **sezilmay qoladi**.

drop_duplicates — olib tashlash

```
df = df.drop_duplicates()
df = df.drop_duplicates(subset=['email'], keep='first')

# Qaysi nusxa qolishini nazorat qilish -- avval saralab
df = df.sort_values('yangilangan_vaqt', ascending=False) \
       .drop_duplicates(subset=['mijoz_id'], keep='first')
```

Amaliy misol: eng so'nggi yozuvni saqlash

Mijoz ma'lumoti bir necha marta yangilangan bo'lsa, oxirgisi eng ishonchli. Avval `sort_values` bilan vaqt bo'yicha saralab, keyin `drop_duplicates(keep='first')` — shu tartibda **eng yangisi** saqlanadi.

Diqqat: mantiqiy dublikat

"Toshkent" va "toshkent" pandas uchun **boshqa-boshqa** qiymat. Dublikat tekshiruvidan oldin matnni tozalash (keyingi bo'lim) kerak bo'lishi mumkin.

? Nechta noyob mijoz bor?

Jadvalda 4 ta yozuv bor:

ism	email	shahar
Aziza Karimova	aziza@mail.uz	Toshkent
Aziza Karimova	aziza@mail.uz	TOSHKENT
A. Karimova	aziza@mail.uz	Toshkent
Bekzod Yusupov	bekzod@mail.uz	Samarqand

? Nechta noyob mijoz bor?

Jadvalda 4 ta yozuv bor:

ism	email	shahar
Aziza Karimova	aziza@mail.uz	Toshkent
Aziza Karimova	aziza@mail.uz	TOSHKENT
A. Karimova	aziza@mail.uz	Toshkent
Bekzod Yusupov	bekzod@mail.uz	Samarqand

Javob: Katta ehtimol 2 ta, lekin duplicated() bu jadvalda 0 ni ko'rsatadi!

Uchta "Aziza" yozuvi harf katta-kichikligi va ism formatida farq qiladi — pandas ularni **uchta boshqa** qator deb hisoblaydi. Bu — **yashirin dublikat**, va u faqat matn tozalangandan keyin ko'rinadi.

Ma’lumot tiplarini to’g’irlash

1. Nega tozalash dolzarb
2. Missing values bilan ishlash: dropna va fillna
3. Dublikat qatorlar
4. Ma’lumot tiplarini to’g’irlash
5. Outlierlarni aniqlash va tozalash
6. Matnni tozalash: strip, replace, regex
7. Konsistentlik: hammasini birlashtirish

Tip nega noto’g’ri bo’lib qoladi

```
df.dtypes
# narx object <- SON bo'lishi kerak edi!
# soni object
# sana object <- SANA bo'lishi kerak edi!
# faol object <- True/False bo'lishi kerak edi!
```

Sabablari

- Narxda bo’shliq yoki valyuta belgisi: `"12 000 so’m"`
- Bitta qatorda matn aralashgan: `"yo’q"`, `"noma’lum"`
- Sana matn ko’rinishida: `"15-mart-2026"`
- Boolean `"ha"`/`"yo’q"` yoki `"1"`/`"0"` matn sifatida

Eslatma (2-ma’ruzadan): `object` deyarli doim “bu ustunda matn bor” degani.

astype — to’g’ridan-to’g’ri o’zgartirish

```
df['soni'] = df['soni'].astype(int)
df['narx'] = df['narx'].astype(float)
df['faol'] = df['faol'].astype(bool)
df['kod'] = df['kod'].astype(str) # SON emas, matn qilib saqlash

# Kategoriyali ustun -- xotira tejaydi (2-ma'ruzada ko'rgandek)
df['shahar'] = df['shahar'].astype('category')
```

astype qachon ishlamaydi

```
ValueError: could not convert string to float: '12 000'
```

`astype` **xato qiladi** — `"12 000"` ichidagi bo’shliqni tushunmaydi va to’g’ridan-to’g’ri xato beradi. Avval matnni tozalash kerak.

to_numeric va to_datetime — aqlliroq usul

```
# Avval tozalash, keyin o'zgartirish
df['narx'] = df['narx'].str.replace(' ', '').str.replace(',', '.')
df['narx'] = pd.to_numeric(df['narx'])

# errors parametri -- muvaffaqiyatsiz qiymatlar bilan nima qilish
pd.to_numeric(df['narx'], errors='raise') # standart -- xato to'xtatadi
pd.to_numeric(df['narx'], errors='coerce') # muvaffaqiyatsiz -> NaN
pd.to_numeric(df['narx'], errors='ignore') # o'zgartirmay qoldiradi

# Sana bilan
df['sana'] = pd.to_datetime(df['sana'])
df['sana'] = pd.to_datetime(df['sana'], format='%d-%m-%Y')
df['sana'] = pd.to_datetime(df['sana'], errors='coerce')
```

errors=’coerce’ — eng foydali usul

Noto’g’ri qiymatlarni `NaN` qilib, dasturni to’xtatmaydi. Keyin `isna()` bilan qaysilari muammoli ekanini ko’rasiz.

Amaliy misol: aralash narx ustuni

```
narxlar = pd.Series(['4500', '12 000', '8,000', 'noma\'lum', '3000 som'])

# 1-urinish -- ishlamaydi
# pd.to_numeric(narxlar) -> ValueError

# To'g'ri yo'l: tozalash zanjiri
tozalangan = (narxlar
    .str.replace(' ', '', regex=False)
    .str.replace(',', '', regex=False)
    .str.replace('som', '', regex=False)
    .str.strip())

natija = pd.to_numeric(tozalangan, errors='coerce')
# [4500.0, 12000.0, 8000.0, NaN, 3000.0]

natija.isna().sum() # 1 -- "noma'lum" ustunini topib berdi
```

Naqsh: tozalash — zanjir (`.str...str...str`), oxirida `to_numeric(errors='coerce')`.

Outlierlarni aniqlash va tozalash

1. Nega tozalash dolzarb
2. Missing values bilan ishlash: dropna va fillna
3. Dublikat qatorlar
4. Ma’lumot tiplarini to’g’irlash
5. Outlierlarni aniqlash va tozalash
6. Matnni tozalash: strip, replace, regex
7. Konsistentlik: hammasini birlashtirish

Outlier nima va nima outlier emas?

Ta’rif

Outlier (chetdagi qiymat) — qolgan ma’lumotdan sezilarli farq qiladigan kuzatuv. Ikki xil sababi bo’lishi mumkin: **xato** yoki **haqiqiy noodatiy hodisa**.

Ehtimol — XATO

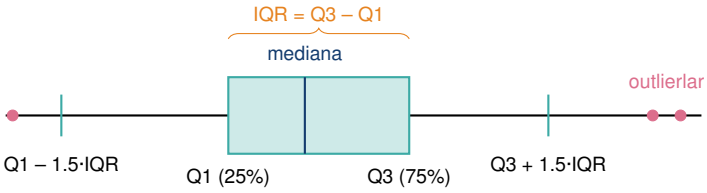
Yosh $= 999$
Bo’y $= 2.10$ metr (odam uchun)
Narx $= -500$
Baho $= 150$ (100 balldan)

Ehtimol — HAQIQIY

Daromad \$50,000,000 (miliarder)
Bitta buyurtma 10,000 dona
Yer silkinishi 8.5 ball

Avtomatik o’chirmang. Har outlier — **tekshirish uchun signal**, tashlab yuborishga shoshilmang!

IQR usuli — keng tarqalgan usul




```
Q1 = df['narx'].quantile(0.25)
Q3 = df['narx'].quantile(0.75)
IQR = Q3 - Q1
pastki = Q1 - 1.5 * IQR
yuqori = Q3 + 1.5 * IQR
outlierlar = df[(df['narx'] < pastki) | (df['narx'] > yuqori)]
```

Z-score usuli — normal taqsimot uchun

```
ortacha = df['narx'].mean()
std = df['narx'].std()
df['z_score'] = (df['narx'] - ortacha) / std

# |z| > 3 -- odatda outlier deb qaraladi
outlierlar = df[df['z_score'].abs() > 3]

# scipy bilan qisqaroq
from scipy import stats
df['z_score'] = stats.zscore(df['narx'])
```

IQR yoki Z-score?

`Z-score` — ma'lumot **normal taqsimotga** yaqin bo'lganda ishonchli.
`IQR` — **qiyshiq** taqsimotlarda (masalan, maosh, narx) barqarorroq, chunki medianaga asoslangan.

Diqqat

`mean` va `std`ning o'zi outlierlardan **ta'sirlanadi** — bitta juda katta qiymat chegarani "siljitib" yuborishi mumkin.

Outlier bilan nima qilish — to’rtta yo’l

```
# 1. O'chirish -- faqat XATO ekaniga ISHONCH bo'lsa
df = df[(df['narx'] >= pastki) & (df['narx'] <= yuqori)]

# 2. Cheklash (capping/winsorizing) -- ma'lumotni yo'qotmasdan
df['narx'] = df['narx'].clip(lower=pastki, upper=yuqori)

# 3. NaN qilib, keyin fillna strategiyasini qo'llash
df.loc[df['narx'] > yuqori, 'narx'] = np.nan

# 4. Alohida belgi qo'yib, SAQLAB QOLISH
df['outlier_flag'] = (df['narx'] < pastki) | (df['narx'] > yuqori)
```

Qaror qabul qilishdan oldin — doim tekshiring

`df[df['z_score'].abs() > 3][['mahsulot','narx','shahar']]` — outlierlarni **ko’zdan kechiring**, faqat raqamga ishonmang.

Amaliy misol: uy narxlari

Toshkentdagi uy narxlari (mln so'm): `450, 520, 480, 510, 495, 8900, 505`

Avtomatik yechim (o'ylamasdan)

"8900 outlier, o'chiramiz." — Lekin bu **markazdagi** qimmat uy bo'lishi mumkin, xato emas!

To'g'ri yondashuv

1. Manbani tekshiring: shu uyning **maydoni** va **joylashuvi** qanday?
2. Agar maydon ham katta (500 m^2) bo'lsa — **haqiqiy**, saqlang
3. Agar maydon oddiy (60 m^2) bo'lsa — ehtimol **qo'shimcha nol** kiritilgan (89.0 o'rniga 8900) — **xato**, tuzating

Xulosa: statistika **shubha uyg'otadi**, lekin **qaror** kontekst bilan qabul qilinadi.

Matnni tozalash: strip, replace, regex

1. Nega tozalash dolzarb
2. Missing values bilan ishlash: dropna va fillna
3. Dublikat qatorlar
4. Ma’lumot tiplarini to’g’irlash
5. Outlierlarni aniqlash va tozalash
6. Matnni tozalash: strip, replace, regex
7. Konsistentlik: hammasini birlashtirish

.str — matn ustunlarining maxsus vositasi

```
df['shahar'] = pd.Series([' Toshkent ', 'SAMARQAND', 'buxoro', 'Toshkent'])

df['shahar'].str.strip() # bo'shliqlarni olib tashlash
df['shahar'].str.lower() # kichik harf
df['shahar'].str.upper() # katta harf
df['shahar'].str.title() # Har Bir So'z Bosh Harf Bilan
df['shahar'].str.len() # har matnning uzunligi
```

Standart tozalash zanjiri

```
df['shahar'] = df['shahar'].str.strip().str.lower().str.title()
# natija: 'Toshkent', 'Samarqand', 'Buxoro', 'Toshkent'
```

Eslatma: shu qatordan keyin “Toshkent” va “ Toshkent ” endi **bir xil** — dublikat va guruhlash to’g’ri ishlaydi.

replace — almashtirish

```
# Aniq qiymatni almashtirish
df['shahar'] = df['shahar'].replace('Toshkentt', 'Toshkent')

# Lug'at bilan -- bir necha xatoni bir yo'la
tuzatish = {'Toshkentt': 'Toshkent', 'samarqand': 'Samarqand',
            'buxoroo': 'Buxoro'}
df['shahar'] = df['shahar'].replace(tuzatish)

# Matn ICHIDAGI qismni almashtirish
df['narx'] = df['narx'].str.replace('so\'m', '', regex=False)
df['narx'] = df['narx'].str.replace(' ', '', regex=False)
```

replace() vs str.replace() – ikkisi boshqa narsa

`df['x'].replace(a, b)` — **butun qiymat** `a` ga teng bo'lsa, `b` ga almashtiradi.
`df['x'].str.replace(a, b)` — matn **ichidagi** `a` qismini topib almashtiradi.

Regex — naqsh bo'yicha qidirish

Regex nima

Regular expression (regex) — matnda **naqsh** tasvirlaydigan mini-til. "Aniq shu so'z" emas, "shunga o'xshash hamma narsa"ni topadi.

```
telefonlar = pd.Series(['+998901234567', '90-123-45-67',
                        '(90) 123 45 67', 'noaniq'])

# Faqat raqamlarni qoldirish
telefonlar.str.replace(r'\D', '', regex=True)
# ['998901234567', '90123 4567', ...]

# Naqshga mosligini tekshirish
telefonlar.str.match(r'^\+998\d{9}$') # to'liq formatga mos keladimi?
telefonlar.str.contains(r'\d{7,}') # kamida 7 ta ketma-ket raqam bormi?

# Naqsh bo'yicha AJRATIB olish
telefonlar.str.extract(r'(\d{2})[\s\-]?(\d{3})[\s\-]?(\d{2})[\s\-]?(\d{2})')
```

Regex tez ma’lumotnomasi

Belgi	Ma’nosi	Misol
`\d`	Bitta raqam	`\d{4}` → 2026
`\D`	Raqam BO’LMAGAN belgi	telefon tozalashda
`\s`	Bo’shliq	`\s+` → ketma-ket bo’shliqlar
`+`	Bir yoki ko’proq marta	`\d+` → istalgan uzunlikdagi son
`*`	Nol yoki ko’proq marta	ixtiyoriy takrorlanish
`^ $`	Qator boshi / oxiri	`^998` → 998 bilan boshlansin
`()`	Guruhlash (ajratib olish)	`extract` bilan birga
`[A-Za-z]`	Harflar diapazoni	faqat lotin harflari

Maslahat

Murakkab regex yozishdan oldin **regex101.com** saytida sinab ko’ring — har qismi tushuntirilib boradi.

Amaliy misol: email va telefon validatsiyasi

```
mijozlar = pd.DataFrame({
    'email': ['aziza@mail.uz', 'notogri-email', 'bekzod@gmail.com',
              'yana@ham@xato.uz'],
    'telefon': ['+998901234567', '901234567', '99-999', '+998971112233']
})

email_naqsh = r'^[\w\.-]+@[\w\.-]+\.\w+$'
mijozlar['email_togri'] = mijozlar['email'].str.match(email_naqsh)

tel_naqsh = r'^\+998\d{9}$'
mijozlar['tel_togri'] = mijozlar['telefon'].str.match(tel_naqsh)

# Xato yozuvlarni ko'rish
mijozlar[~mijozlar['email_togri'] | ~mijozlar['tel_togri']]
```

Amaliy foyda: noto'g'ri email'ga xabar yuborish yoki noto'g'ri raqamga SMS xarajatini oldindan tutadi.

Konsistentlik: hammasini birlashtirish

1. Nega tozalash dolzarb
2. Missing values bilan ishlash: dropna va fillna
3. Dublikat qatorlar
4. Ma’lumot tiplarini to’g’irlash
5. Outlierlarni aniqlash va tozalash
6. Matnni tozalash: strip, replace, regex
7. Konsistentlik: hammasini birlashtirish

Konsistentlik nima

Ta’rif

Konsistentlik — ma’lumotning turli qismlari bir-biriga **mantiqan zid kelmasligi**. Har bir qiymat alohida to’g’ri ko’rinsa ham, birgalikda **ziddiyatli** bo’lishi mumkin.

Ziddiyat misollari

- `tugilgan_sana` > `ishga_kirgan_sana`
- `yosh` = 17, lekin `lavozim` = “direktor”
- `shahar` = “Toshkent”, `viloyat` = “Farg’ona”
- `boshlanish_vaqti` > `tugash_vaqti`

Nega bu maxsus e’tibor talab qiladi

Har ustun `isna()` va `astype` testidan **muvaffaqiyatli** o’tishi mumkin — muammo faqat ustunlarni **birga** ko’rganda ko’rinadi.

Konsistentlikni tekshirish naqshlari

```
# 1. Sanalar mantig'i
xato_sana = df[df['tugilgan_sana'] > df['ishga_kirgan_sana']]

# 2. Diapazon tekshiruvi
xato_yosh = df[(df['yosh'] < 0) | (df['yosh'] > 120)]
xato_foiz = df[(df['chegirma'] < 0) | (df['chegirma'] > 100)]

# 3. Geografik moslik (lug'at bilan)
shahar_viloyat = {'Toshkent': 'Toshkent', 'Samarqand': 'Samarqand',
                  'Buxoro': 'Buxoro'}
df['kutilgan_viloyat'] = df['shahar'].map(shahar_viloyat)
nomos = df[df['kutilgan_viloyat'] != df['viloyat']]

# 4. Mantiqiy tartib: boshlanish < tugash
xato_vaqt = df[df['boshlanish'] > df['tugash']]
```

Umumiy naqsh: har safar **shart yozib**, uni buzadigan qatorlarni `df[shart]` bilan ko'rish.

Amaliy misol: xodimlar bazasi

```
xodimlar = pd.DataFrame({
    'ism': ['Aziza', 'Bekzod', 'Dilnoza'],
    'tugilgan_sana': pd.to_datetime(['1995-03-01','2010-05-01','1988-01-01']),
    'ishga_kirgan': pd.to_datetime(['2020-01-01','2021-01-01','2015-06-01']),
    'maosh': [8_000_000, 6_000_000, -500_000],
})

xodimlar['yosh_ishga_kirganda'] = (
    (xodimlar['ishga_kirgan'] - xodimlar['tugilgan_sana']).dt.days / 365.25
)

# Tekshiruvlar zanjiri
juda_yosh = xodimlar[xodimlar['yosh_ishga_kirganda'] < 18]
manfiy_maosh = xodimlar[xodimlar['maosh'] < 0]

print(f"Yosh chegarasi buzilgan: {len(juda_yosh)}")
print(f"Manfiy maosh: {len(manfiy_maosh)}")
```

Bekzod 2021-yilda ishga kirganda **10 yoshda** bo'lgan — sana xato kiritilgan.

Tozalashni tekshirish uchun yagona funksiya

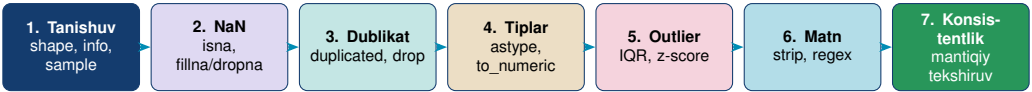
```
def tozalik_hisoboti(df):
    print(f"Qatorlar: {len(df)}, Ustunlar: {df.shape[1]}")
    print(f"\nBo'sh qiymatlar (ustun bo'yicha):")
    print(df.isna().sum()[df.isna().sum() > 0])
    print(f"\nDublikat qatorlar: {df.duplicated().sum()}")
    print(f"\nTiplar:\n{df.dtypes}")
    for ustun in df.select_dtypes(include='number').columns:
        print(f"{ustun}: min={df[ustun].min()}, max={df[ustun].max()}")

tozalik_hisoboti(df)
```

Foydali

Yangi fayl bilan ishlashni shu funksiyadan boshlang — 3-ma'ruzadagi `info()`/`describe()` tartibiga **tozalikka moslashgan** qo'shimcha.

Tozalash ish oqimi — yakuniy sxema



Tartib nima uchun muhim

Tip to'g'irlanmasdan outlier qidirish mumkin emas (matn ustida `quantile` ishlamaydi). Matn tozalanmasdan dublikat tekshirish chalg'itadi. **Ketma-ketlik — juda muhim omil!**

Xulosa:

Yodda saqlang!

1. Tozalash — loyihaning **eng katta** qismi, qo'shimcha emas
2. `NaN == NaN` → `False`. Har doim `isna()` ishlating
3. `dropna` — oson, lekin **ma'lumot yo'qotadi**. Avval necha foiz yo'qolishini o'lchang
4. `astype` aqlli emas — avval tozalang, keyin o'zgartiring, yoki `to_numeric(errors='coerce')`
5. Outlier — **signal**, hukm emas. Kontekstsiz o'chirmang
6. Matn tozalash (`strip`, `lower`) — dublikat va guruhlashdan **oldin** keladi
7. Konsistentlik — ustunlarni **birga** ko'rish, alohida emas

Boshlovchilardagi ko’p uchraydigan xatolar

Xato	To’g’ri yondashuv
`== None` bilan bo’sh qiymat qidirish	`isna()` / `notna()` ishlatish
`dropna()` ni o’ylamasdan qo’llash	Avval necha foiz yo’qolishini hisoblash
`duplicated()` ni parametrsiz ishlatish	`subset=` bilan mos ustunlarni ko’rsatish
Matn tozalashdan oldin dublikat qidirish	Avval `strip().lower()`, keyin `duplicated()`
Har outlierni avtomatik o’chirish	Avval manbani tekshirish, keyin qaror
`astype(float)` ni to’g’ridan-to’g’ri matnga qo’llash	`to_numeric(errors='coerce')` bilan boshlash
Ustunlarni faqat alohida tekshirish	Konsistentlik — ustunlar orasidagi mantiq

Universal maslahat: har tozalash qadamidan keyin `df.shape` va `tozalik_hisoboti(df)` bilan natijani tekshiring.

Uy vazifasi

1-topshiriq: diagnostika

Ataylab “buzilgan” dataset yarating (kamida 30 qator): bo’sh qiymatlar, dublikatlar, noto’g’ri tiplar, outlierlar va turli formatdagi matn bo’lsin. `tozalik_hisoboti()` funksiyasi bilan muammolarni hujjatlashtiring.

2-topshiriq: tozalash zanjiri

O’sha datasetni birma-bir tozalang: mos strategiya bilan NaN’larni hal qiling, dublikatlarni olib tashlang, tiplarni to’g’irlang, IQR usuli bilan outlierlarni toping (o’chirmasdan, faqat belgilang).

3-topshiriq: konsistentlik

Kamida 3 ta mantiqiy shart yozing (masalan, sanalar tartibi, diapazon chegaralari) va ularni buzadigan qatorlarni toping. Har biri uchun: bu xatomi yoki haqiqiy holatmi — izohlang.

Natijani Jupyter Notebook ko’rinishida GitHub repozitoriyangizga yuklang.

Resurslar

Hujjatlar

- **pandas: Working with missing data** — rasmiy hujjat
- **Python for Data Analysis** (McKinney) — 7-bob
- **regex101.com** — regex sinov maydoni
- **missingno** kutubxonasi — bo'shliqlarni vizuallashtirish

Amaliyot

- **Kaggle: Data Cleaning** — bepul mikro-kurs
- **Messy datasets** (GitHub qidiruvi) — mashq uchun
- **Kaggle Datasets** — real ifloz ma'lumot

Maslahat

Toza ko'rinadigan dataset topish qiyin — va bu **yaxshi**. Har birida kamida bitta muammo bor, uni qidirish — eng yaxshi mashq.

Keyingi darsda

Mavzu: ma’lumotlarni birlashtirish va qayta shakllantirish

- `concat` — jadvallarni ustma-ust va yonma-yon ulash
- `merge` va `join` — kalit bo’yicha birlashtirish, join turlari
- `pivot`, `melt`, `stack`/`unstack` — jadval shaklini o’zgartirish

Darsga tayyorgarlik

Ma’lumot tozalash mavzusini takrorlab, uy vazifasidagi amaliy topshiriqlarni bajarib qo’yish

Savollar ?

E’tiboringiz uchun rahmat! Kuningiz xayrli o’tsin!